

* For every class there is a parent class called Object class.

* There are 8 important methods in Object class.

- | | | |
|-------------------------|----------------|--|
| 1. equals() | 5. finalise() | [Control 0 → to see objects]
This method is deprecated. |
| 2. hashCode() | 6. notify() | |
| 3. toString() | 7. wait() | - Threads. |
| 4. clone() - Interface. | 8. notifyAll() | |

* getClass() is common method for all classes.

* ctrl+A, ctrl+shift+f → to get formatted.
(select all)

1. equals()

* It is used to compare two objects.

Ex: obj1, obj2

obj1.equals(obj2) ⇒ obj1 becomes this reference.

* It is an instance method.

Ex: package com;

public class EqualsMethod {

int age;

String name;

Person (int age, String name) {

this.age = age;

this.name = name;

}

public static void main (String[] args) {

Person person1 = new Person();

person1.age = 20;

person1.name = "Gayatri";

Person person2 = new Person(20, "Ravi");

boolean isSame = person1.equals(person2); ⇒ [ctrl+I] (shortcut)

System.out.println(isSame);

Output: false.

(Compares the references)

* Person person3 = new Person();

person3.setAge(30);

person3.setAge("Ravi");

boolean isSame2 = person2.equals(person3);

System.out.println(isSame2); → output: False.

[* If we not override equals() method, it will check reference but not check content]

```
}  
public void setAge() {
```

```
    this.age = age;
```

```
}
```

```
public int getAge() {
```

```
    return this.age;
```

```
}
```

```
public void setName() {
```

```
    this.name = name;
```

```
}
```

```
public String getName() {
```

```
    return this.name;
```

```
}
```

@Override;

```
public boolean equals(Object obj) {
```

```
    if (this == obj)
```

```
        return true;
```

```
    if (this == null)
```

```
        return false;
```

```
    if (getClass() != obj.getClass())
```

```
        return false;
```

```
    Person other = (Person) obj;
```

```
    return this.age == other.age && Object.equals(name, other.name);
```

```
        this.age  
        ↓  
        (person1)  
        person  
        (person2)
```

```
name.equals(person.name);
```

[alt + shift + x]

↓
to change the
variable in all
places]

[* (ctrl + shift + i)]

↓
to see assigned
values when we
debugging the code]

output:

false

true

(due to override)
the equals method]

2. hashCode()

- * It is a unique value, which are generated used to identify objects uniquely.
- * It is not memory address in Java.

Ex: `sysout(person1.hashCode());` output: 1159190947
`sysout(person2.hashCode());` output: 925858445
`sysout(person3.hashCode());` output: 798154996.

- * It will generate a random value.

- * To get the same value to same content in `hashCode()` we need to override the `hashCode` method.

@ Override

```
public int hashCode() {  
    return Objects.hash(this.age, this.name);  
}
```

person1
person2
person3

output: 67657188
90268 ✓
90268 ✓

- * `hash()` is a static method, because it is calling with object class name.

- * HashCode and equals Contract:

- * When two objects are equal, they should have equal hashCode.

- * When two hashcodes are equal, they need not be an equal objects.

- * This is called Hash Collision.

→ This happens when by mistake same hash is generated.

→ `Math.random(1000)` → gives a random value below 1000, but there is a probability to repeat, this is called hash Collision.

* 3. toString():

* `System.out.println(person1);` → output: com.Person@4085de4

* When we print an object, internally `toString()` method will be called.

* `@Override`
`public String toString() {`
`return getClass().getName() + "@" + Integer.toHexString(hashcode());`
`}`

`@Override`

```
public String toString() {  
    // return "Hi...";  
    return this.age + " " + this.name;  
}
```

`System.out.println(person1);` output: Hi...

`System.out.println(person1);` output: 20 Gayatri

`System.out.println(person2);` output: 30 Ravi

* `toString()` is a static method in arrays so it is called with class name i.e. `Arrays.toString()`.

class: 23

17/05/2025

finalise():

* This is used by Garbage Collector.

* When garbage collector is invoked `finalise()` method be called by JVM internally.

* It is deprecated currently.